# Apuntes para el examen

Despliegue de aplicaciones web · Unidad 1 · Git, GitHub Pages y GitHub Actions

## Índice (pulsa en un apartado para ir directo a él)


**1. El examen de un vistazo . . . 2**

Orden de trabajo en cada pregunta . . . 2

**2. Qué hacer según lo que te pidan . . . 2**

**3. Token y configuración . . . 4**

El token . . . 4

git config . . . 4

**4. Git básico . . . 5**

Las tres zonas . . . 5

Ciclo normal de trabajo . . . 5

Deshacer el último commit (actividades iniciales) . . . 5

**5. Ramas y tags . . . 5**

**6. Conectar con GitHub: remote y push . . . 6**

Qué es un remote . . . 6

Errores frecuentes con el push . . . 6

**7. GitHub Pages . . . 6**

Si la web no abre o no se ve lo último . . . 7

**8. GitHub Actions . . . 8**

Las piezas del workflow . . . 8

Plantilla completa (ejercicio PyInstaller), comentada . . . 9

Por qué hacen falta artefactos y needs . . . 10

Cómo se ejecuta el script del ejercicio (viene explicado en su cabecera) . . . 10

Comprobar que funciona . . . 10

Errores típicos al escribir el workflow . . . 10

Dónde buscar otra versión de Python . . . 10

**9. Variaciones típicas de Actions . . . 11**

a) Instalar los paquetes desde requirements.txt . . . 11

b) Una rama con otra versión de Python . . . 11

c) Una rama que ejecuta el script solo con un parámetro . . . 11

d) Probar varias versiones de Python a la vez (matrix) . . . 11

**10. Checklist final del examen . . . 12**

# 1. El examen de un vistazo

El examen es **práctico**: lo que se ha hecho en clase con git, GitHub Pages y GitHub Actions. Cada pregunta lleva asociado un **repositorio**, y **crear el repositorio es parte del examen**. En **Pages** el profesor da una web y hay que ponerla online. En **Actions** el profesor da unos ficheros, explica qué hacen y hay que preparar un action para lo que pida.

**Ojo:** Lo que **no** aparece en los apuntes de clase está marcado como *(fuera de apuntes)* o *(de las prácticas)*.

## Orden de trabajo en cada pregunta

| Paso | Qué hacer |
|---|---|
| 1 | Crear el **token** en GitHub (una sola vez) y tenerlo copiado fuera del repo. |
| 2 | `git config` si lo piden (nombre y correo). |
| 3 | Crear en GitHub el repositorio **vacío** (sin README, sin .gitignore, sin licencia). |
| 4 | En la terminal: carpeta de trabajo, copiar los archivos del profesor, `git init`, `git add`, `git commit`. |
| 5 | `git branch` para ver el nombre de la rama. `git remote add origin URL`. |
| 6 | `git push -u origin RAMA` (usuario y token). Después ya basta `git push`. |
| 7 | **Pages:** Settings → Pages → elegir esa rama. **Actions:** mirar la pestaña Actions. |

# 2. Qué hacer según lo que te pidan

Busca aquí la frase del enunciado y mira qué usar. La columna **Ver** es la sección donde está explicado.

## Git

| Si te piden... | Haz / usa... | Ver |
|---|---|---|
| Poner tu nombre y correo | `git config --global user.name "X"` y `git config --global user.email "x@y.com"` | 3 |
| Comprobar la configuración | `git config --list` | 3 |
| Crear un repositorio | Crear la carpeta, entrar con `cd` y `git init`. Si hay que subirlo, crear también el repo vacío en GitHub. | 4 |
| Ver el estado del repo | `git status` | 4 |
| Ver qué has cambiado antes de guardarlo | `git diff` (si ya hiciste add: `git diff --staged`) | 4 |
| Guardar un archivo nuevo o modificado | `git add archivo` y después `git commit -m "texto"` | 4 |
| Guardar con un solo comando (archivo ya rastreado) | `git commit -am "texto"` | 4 |
| Ver el historial / cuántos commits hay | `git log --oneline` | 4 |
| Borrar un archivo del repo | `git rm archivo` y `git commit` | 4 |
| Deshacer el último commit | `git reset --soft HEAD~1` (conserva cambios) o `--hard` (los borra) | 4 |
| Ver o crear ramas | `git branch` (ver) · `git switch -c nombre` (crear y entrar) | 5 |
| Fusionar una rama | Ir a la rama destino con `git switch` y `git merge rama` | 5 |
| Marcar la entrega con un tag | `git tag -a nombre -m "texto"` y subirlo con `git push origin nombre` | 5 |
| Conectar con GitHub | `git remote add origin URL` | 6 |
| Subir por primera vez | `git push -u origin RAMA` (usuario + token) | 6 |
| Subir cambios nuevos | `git add`, `git commit`, `git push` | 6 |
| Ver a qué repo apunta la carpeta | `git remote -v` | 6 |

## GitHub Pages

| Si te piden... | Haz / usa... | Ver |
|---|---|---|
| Poner una web online | Repo `TUUSUARIO.github.io` vacío → subir la web con git → Settings → Pages → elegir la rama. | 7 |
| Comprobar que funciona | Abrir `https://TUUSUARIO.github.io` (esperar 1-2 min, Ctrl+F5). | 7 |
| Subir una foto o archivo nuevo a la web | Copiarlo a la carpeta, `git add`, `git commit`, `git push`. | 7 |

### GitHub Actions

| Si te piden... | Haz / usa... | Ver |
|---|---|---|
| Crear un workflow | Archivo `.github/workflows/nombre.yml` subido a GitHub. | 8 |
| Que se ejecute cada vez que se hace push | `on: push` | 8 |
| Que solo se ejecute en una rama | `on:` → `push:` → `branches:` con el nombre de la rama. | 8 |
| Ejecutar en Linux / Windows / macOS | `runs-on: ubuntu-latest` / `windows-latest` / `macos-latest` (un job por sistema). | 8 |
| Bajar el código del repo a la máquina | `uses: actions/checkout@v4` | 8 |
| Instalar Python / usar otra versión | `uses: actions/setup-python@v5` con `with: python-version: '3.12'` | 8, 9 |
| Probar varias versiones de Python | `strategy: matrix:` con la lista de versiones. | 9 |
| Instalar un paquete | `run: pip install paquete` | 8 |
| Instalar desde requirements.txt | `run: pip install -r requirements.txt` (y subir el archivo al repo) | 9 |
| Ejecutar un comando o script | `run: comando` | 8 |
| Ejecutar un script con un parámetro | `run: python euro.py USD` | 9 |
| Que un job espere a otro | `needs: nombre-del-job` | 8 |
| Usar en un job el archivo que creó otro | `upload-artifact` en el primero, `download-artifact` en el segundo y `needs`. | 8 |
| Otra configuración en otra rama | Crear la rama, cambiar el workflow, `commit` y `git push -u origin rama`. | 9 |
| Comprobar que funciona | Pestaña Actions del repo: verde bien, rojo falla (leer el paso en rojo). | 8 |

# 3. Token y configuración

## El token

Es una clave que GitHub te da en lugar de la contraseña. Sin él, el `git push` falla con *"Password authentication is not supported"*. Esto lo explicó el profesor, no está en los PDF de clase.

- Crearlo: GitHub → Settings → Developer settings → Personal access tokens, con **todos los permisos** (como dijo el profesor).
- Copiarlo y guardarlo **fuera** de cualquier repositorio. Nunca en un README, ni en un PDF de apuntes, ni dentro de la URL del remote.
- Usarlo en el `git push` cuando pida *Username* y *Password*. Tu usuario de GitHub es el usuario y el token va en la contraseña (el profesor decía que se puede poner el token en los dos campos). Al pegarlo en la contraseña no se ve nada: es normal.
- Si sale *Invalid username or token*: usuario mal escrito, token mal pegado o token caducado. Crea uno nuevo.

## git config

Fija tu identidad. Tres niveles (cada uno pisa al anterior): `--system` (todos los usuarios), `--global` (tu usuario) y sin opción (solo ese repositorio).

```
git config --global user.name "TuNombre"
git config --global user.email "tucorreo@ejemplo.com"
git config --list            # comprobar lo guardado
```

# 4. Git básico

## Las tres zonas

Directorio de trabajo (tus archivos) → `git add` → **zona de preparación** (staging) → `git commit` → **repositorio** (historial). `git status` te dice en cuál está cada archivo.

| Comando | Para qué sirve |
|---|---|
| `git init` | Convierte la carpeta actual en repositorio (crea el .git). |
| `git status` | Estado: archivos sin rastrear (*Untracked*), modificados o preparados (*Changes to be committed*). |
| `git add archivo` | Prepara un archivo. `git add .` prepara todo lo de la carpeta. |
| `git commit -m "texto"` | Guarda lo preparado en el historial con ese mensaje. |
| `git commit -am "texto"` | Atajo: `git add` + `git commit` en un paso. **Solo vale para archivos que Git ya rastrea**; un archivo nuevo necesita antes `git add`. |
| `git log --oneline` | Historial resumido, una línea por commit. |
| `git diff` | Enseña lo que has **modificado y aún no has preparado** (líneas nuevas con +, borradas con -). |
| `git diff --staged` | Lo mismo, pero de lo que ya está preparado con `git add` (también `--cached`). |
| `git diff rama1 rama2` | Compara dos ramas. |
| `git rm archivo` | Borra el archivo y lo deja preparado para el commit. |
| `git restore archivo` | Recupera el archivo del último commit (descarta cambios en el directorio). |

## Ciclo normal de trabajo

```
nano archivo.txt              # editar
git status                    # ver que archivo ha cambiado
git diff                      # ver QUE ha cambiado (antes de preparar)
git add archivo.txt           # preparar
git commit -m "Mensaje"       # guardar
git log --oneline             # comprobar
```

**Ojo: git diff vacío** no significa que no hayas tocado nada: si ya hiciste `git add`, usa `git diff --staged`. Y si tu archivo es **nuevo**, `git commit -am` no lo incluye: haz primero `git add`. Si el `git diff` se queda en pantalla, sal con la tecla **q**.

## Deshacer el último commit (actividades iniciales)

```
git reset --soft HEAD~1       # deshace el commit, deja los cambios preparados
git reset --hard HEAD~1       # deshace el commit Y borra los cambios (cuidado)
```

# 5. Ramas y tags

De la práctica Branques i tags. `git branch` lista las ramas y marca con * la actual.

```
git branch                          # listar ramas (* = la actual)
git branch --show-current           # nombre de la rama actual
git switch -c contacte              # crear la rama y moverse a ella
git switch master                   # volver a master
git merge contacte                  # fusionar contacte dentro de la rama actual
git log --oneline --graph --all     # ver el historial con las ramas
git branch -d contacte              # borrar una rama ya fusionada
git tag -a entrega-NOMBRE -m "Entrega de la prueba"   # tag anotado
git show entrega-NOMBRE             # ver el tag
git push origin entrega-NOMBRE      # subir el tag (un git push normal NO lo sube)
```

La rama inicial puede llamarse `master` o `main` según la versión de Git: compárala siempre con `git branch`. Para renombrarla: `git branch -m master`.

# 6. Conectar con GitHub: remote y push

## Qué es un remote

Cada carpeta con un `.git` es un repositorio **independiente**. Para cambiar de repositorio simplemente haces `cd` a su carpeta. El **remote** es la dirección del repositorio de GitHub al que **esa carpeta** sube sus cambios. `origin` es solo el apodo habitual de esa dirección.

| Comando | Para qué sirve |
|---|---|
| `git remote add origin URL` | Añade la dirección del repo de GitHub con el nombre origin. Se hace **una vez** por carpeta. |
| `git remote -v` | Lista las direcciones configuradas. Comprueba a qué repo apunta la carpeta donde estás. |
| `git remote show origin` | Configuración detallada del remote. |
| `git push -u origin RAMA` | Sube la rama y la enlaza con origin. **Solo la primera vez**. |
| `git push` | Sube los commits nuevos (después del primer push con -u). |
| `git clone URL` | Descarga un repo de GitHub a una carpeta nueva (ya trae el remote origin puesto). |
| `git remote set-url origin URL` | *(de las prácticas)* Corrige la dirección si te equivocaste al añadirla. |

```
# Formato de la URL (HTTPS):
git remote add origin https://github.com/TUUSUARIO/NOMBRE-DEL-REPO.git
git push -u origin master        # usa el nombre de rama que te diga git branch
```

## Errores frecuentes con el push

| Mensaje | Causa y solución |
|---|---|
| *Repository not found* | El repo no existe en GitHub, o el nombre/usuario de la URL está mal. Créalo (vacío) o corrige con `git remote -v` y `set-url`. |
| *remote origin already exists* | Ya habías hecho `git remote add`. No pasa nada; comprueba con `git remote -v`. |
| *no upstream branch* | Falta el `-u`: `git push -u origin RAMA`. |
| *src refspec ... does not match any* | El nombre de la rama está mal escrito (o no hay ningún commit todavía). |
| *Invalid username or token* | Usuario mal escrito o token incorrecto/caducado. |
| Pide hacer pull / rechaza el push | El repo de GitHub no estaba vacío (tiene README u otro historial). Créalo vacío. |

# 7. GitHub Pages

El enunciado: *crea un repositorio que se llame usuari.github.io, sube el contenido del comprimido con git y comprueba que la web funciona.*

**El nombre del repositorio tiene que ser exactamente TUUSUARIO.github.io** (por ejemplo, si tu usuario es alvaro, el repo se llama alvaro.github.io).

```
# 1) En GitHub: crear el repo TUUSUARIO.github.io, publico y VACIO
# 2) En la terminal:
mkdir ~/web && cd ~/web
unzip /ruta/al/archivo.zip          # (fuera de apuntes) la web debe quedar en la raiz de la carpeta
ls                                  # index.html, css/, js/ ... directamente aqui, no en una subcarpeta
git init
git branch                          # anotar el nombre de la rama
git add .
git commit -m "Afegit contingut de la web"
git remote add origin https://github.com/TUUSUARIO/TUUSUARIO.github.io.git
git push -u origin master           # (o main, segun git branch) - usuario + token
```

3) Activar Pages: en el repo, **Settings** → **Pages** y elegir la rama donde está la web (la misma a la que has hecho push). Esperar uno o dos minutos y abrir **https://TUUSUARIO.github.io**.

## Si la web no abre o no se ve lo último

- Comprueba que Pages apunta a la **misma rama** a la que subiste (error típico: tener dos ramas, `main` y `master`).
- En la pestaña **Actions** del repo, mira la ejecución *pages build and deployment*: amarillo = publicando, verde = listo.
- Recarga con Ctrl+F5 (caché del navegador).
- Si sale la página 404 de la propia web, Pages funciona pero no encuentra ese archivo: revisa la rama y que el archivo esté subido.
- Para añadir una imagen: copiarla a la carpeta (por ejemplo `img/`), `git add`, `git commit`, `git push`. Git no sube carpetas vacías.

# 8. GitHub Actions

Un **workflow** es un archivo YAML dentro de `.github/workflows/` del repositorio (extensión .yml o .yaml). Se ejecuta automáticamente cuando ocurre el evento que indiques (por ejemplo, un push).

## Las piezas del workflow

| Palabra | Qué es |
|---|---|
| `name` | Nombre del workflow (o del job, o del paso). Es solo una etiqueta para leer mejor los logs. |
| `on` | Evento que lo dispara. `push` con `branches` limita a qué ramas; también existen `pull_request` y `schedule` (cron). |
| `jobs` | Conjunto de trabajos. Cada job tiene un nombre propio (por ejemplo build-linux). Los jobs corren **en paralelo** salvo que uses `needs`. |
| `runs-on` | Sistema donde corre el job: `ubuntu-latest`, `windows-latest`, `macos-latest`. Cada job usa una máquina virtual **nueva**. |
| `steps` | Lista de pasos del job, en orden. Cada paso empieza por un guion `-`. |
| `uses` | Ejecuta una **action ya hecha**: `propietario/repo@versión` (ej. `actions/checkout@v4`). El cheat sheet recomienda indicar siempre la versión. |
| `with` | Parámetros de la action que usa el paso (ej. `python-version: '3.12'`). |
| `run` | Ejecuta **un comando** de terminal, como si lo escribieras tú. |
| `needs` | El job espera a que termine con éxito el job (o los jobs) indicado. Si ese falla, este se salta. |
| `env` | Variables de entorno (en workflow, job o paso). |
| `if` | El paso solo se ejecuta si se cumple la condición. |
| `strategy / matrix` | Repite el job con varias configuraciones (otros sistemas, otras versiones). |

**uses o run, regla rápida:** si el enunciado te da algo con forma **nombre/algo@vX** (actions/checkout@v4), es `uses`. Si te da un **comando** (pip install pyinstaller), es `run`. Las **versiones** (@v4, @v5, Python 3.12) no se inventan: el enunciado las da y se copian **tal cual**.

## Plantilla completa (ejercicio PyInstaller), comentada

```
name: Empaquetat                      # nombre del workflow (etiqueta)

on:                                   # CUANDO se ejecuta
  push:
    branches:
      - master                        # tiene que ser el nombre REAL de tu rama

jobs:
  build-linux:                        # nombre del job (el enunciado lo dicta)
    runs-on: ubuntu-latest            # maquina donde corre
    steps:
      - name: Checkout del repo
        uses: actions/checkout@v4     # baja el codigo del repo a la maquina

      - name: Configurem Python
        uses: actions/setup-python@v5 # instala Python
        with:
          python-version: '3.12'      # parametro de la action

      - name: Instalem pyinstaller
        run: pip install pyinstaller  # comando

      - name: Empaquetem
        run: pyinstaller --onefile comprova_desplegaments.py

      - name: Guardem l'executable    # lo deja en la "estanteria" de artefactos
        uses: actions/upload-artifact@v4
        with:
          name: executable-linux
          path: dist/comprova_desplegaments
```

... y a continuación, en el mismo archivo (mismo nivel que build-linux):

```
  build-macos:                        # igual que build-linux, solo cambia runs-on
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install pyinstaller
      - run: pyinstaller --onefile comprova_desplegaments.py

  build-windows:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install pyinstaller
      - run: pyinstaller --onefile comprova_desplegaments.py

  comprova:
    needs: build-linux                # espera a que termine build-linux
    runs-on: ubuntu-latest
    steps:
      - name: Descarreguem l'executable
        uses: actions/download-artifact@v4
        with:
          name: executable-linux      # mismo nombre que en upload

      - name: Donem permisos d'execucio
        run: chmod +x comprova_desplegaments

      - name: Executem la comprovacio
        run: ./comprova_desplegaments example.com
```

**Ojo:** En esta plantilla he acortado macOS y Windows (pasos sin `name`) para que quepa; funcionan igual. El `chmod +x` y las versiones `@v4` de los artefactos son *(fuera de apuntes)*: el cheat sheet solo nombra `upload-artifact` y `download-artifact` con los parámetros `name` y `path`.

## Por qué hacen falta artefactos y needs

Cada job corre en un ordenador nuevo que se borra al terminar. Por eso `comprova` no ve lo que creó `build-linux`. La solución:

- `upload-artifact`: un job **guarda** un archivo en una "estantería" compartida (con `name` y `path`).
- `download-artifact`: otro job lo **recoge** usando el mismo `name`.
- `needs`: asegura que el job que descarga espera a que el otro haya terminado y subido el archivo.

## Cómo se ejecuta el script del ejercicio (viene explicado en su cabecera)

```
python comprova_desplegaments.py exemple.com
python comprova_desplegaments.py exemple.com httpbin.org/status/500
python comprova_desplegaments.py --fitxer urls.txt --eixida informe.csv
```

Una vez empaquetado, el ejecutable se lanza sin `python`: `./comprova_desplegaments example.com`. El script solo usa la biblioteca estándar, así que no hace falta instalar nada más con pip.

## Comprobar que funciona

- Subir el workflow: `git add .`, `git commit`, `git push`.
- En GitHub, pestaña **Actions**: aparece una ejecución por cada push. **Verde** = todos los jobs bien, **amarillo** = en curso, **rojo** = fallo.
- Si hay rojo: entrar en la ejecución, abrir el job que falla y leer el paso marcado en rojo. Corregir, `git commit` y `git push` otra vez.
- Cada push nuevo genera una ejecución nueva (#1, #2, #3...).

## Errores típicos al escribir el workflow

| Fallo | Cómo evitarlo |
|---|---|
| Erratas en palabras clave (`brach` en vez de `branches`) | Copiar los nombres exactos. Una errata puede romper el workflow o hacer que no se dispare. |
| Indentación | YAML usa **espacios** (2 por nivel), nunca tabuladores. `jobs` va al borde izquierdo y cada job va dentro. |
| Archivo en sitio equivocado | Debe estar en `.github/workflows/nombre.yml` y subido a GitHub. |
| Rama distinta en `branches` | Tiene que coincidir con la rama a la que haces push (`git branch`). |
| Nombres, versiones o comandos distintos del enunciado | Copiar tal cual: nombres de los jobs, `@v4`, `@v5`, Python 3.12, el comando completo. |
| Usar `checkout` en lugar de `setup-python` para instalar Python | Son dos actions distintas: checkout baja el código, setup-python instala Python. |
| Archivo del script no subido | El script tiene que estar en el repo (`git add` y push), si no el workflow no lo encuentra. |

## Dónde buscar otra versión de Python

**Documentación de GitHub: Creación y prueba de Python**
https://docs.github.com/es/actions/tutorials/build-and-test-code/python
Dentro: **Especificación de una versión de Python** (versión exacta, rango o varias con matrix), **Archivo de requisitos** (`pip install -r requirements.txt`) y **Empaquetar datos como artefactos**. En sus ejemplos aparecen `checkout@v6`, `setup-python@v5`, `upload-artifact@v4` y `download-artifact@v5`. Si el enunciado da otra versión, usa la del enunciado.

# 9. Variaciones típicas de Actions

Salen de la práctica del repo **titansmc/actions** (script *euro.py* y un `actions.yml` que instala *requests*). El profesor dará los ficheros y explicará qué hacen; estas son las modificaciones que ya se pidieron.

## a) Instalar los paquetes desde requirements.txt

Crear un archivo `requirements.txt` con **un paquete por línea**, subirlo al repo y cambiar el paso del workflow:

```
# requirements.txt
requests

# en actions.yml, el paso que antes hacia: run: pip install requests
      - name: Instalem paquets
        run: pip install -r requirements.txt
```

Comandos: `git add requirements.txt .github/workflows/actions.yml`, `git commit -m "Uso de requirements.txt"`, `git push`.

## b) Una rama con otra versión de Python

Cada rama tiene **su propia copia** del workflow. Para una rama `test` con Python 3.12: crea la rama desde la principal, cambia la versión y súbela.

```
git switch -c test                      # crear la rama test y entrar
nano .github/workflows/actions.yml      # cambiar:  python-version: '3.12'
git add .github/workflows/actions.yml
git commit -m "Python 3.12 en la rama test"
git push -u origin test
```

## c) Una rama que ejecuta el script solo con un parámetro

Vuelve primero a la rama principal (para no arrastrar lo de `test`) y crea `development`. El script acepta monedas como parámetro (`python euro.py USD`).

```
git switch main                         # (o master) volver a la principal
git switch -c development
nano .github/workflows/actions.yml      # cambiar el paso a:  run: python euro.py USD
git add .github/workflows/actions.yml
git commit -m "euro.py solo para USD"
git push -u origin development
```

## d) Probar varias versiones de Python a la vez (matrix)

```
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
```

El job se repite una vez por cada valor de la lista. El cheat sheet lo explica con `strategy` y `matrix`.

## e) Cosas que se comprueban siempre después

- Que la rama del `on: push: branches` (si lo hay) incluya la rama donde has subido el workflow.
- En la pestaña **Actions**, que aparezca una ejecución por cada rama a la que has hecho push y que esté en verde.
- Con `git log --oneline` y `git status` en cada paso, para no perderte entre ramas.

# 10. Checklist final del examen

## Git y GitHub (cada pregunta)

- Token creado y copiado fuera del repo.
- `git config` hecho si lo piden.
- Repo creado en GitHub, **vacío** y con el nombre exacto que pide el enunciado.
- Archivos del profesor en la carpeta, `git init`, `git add`, `git commit -m`.
- `git branch` para ver la rama; `git remote add origin URL` con la URL correcta (`git remote -v` para comprobar).
- `git push -u origin RAMA`: usuario y token. Comprobar en GitHub que los archivos aparecen.

## Pages

- Repo `TUUSUARIO.github.io`, web en la raíz del repo.
- Settings → Pages → misma rama que la del push. Abrir la URL y comprobarlo.

## Actions

- Leer qué hace el script/los ficheros y cómo se usa (está explicado en el propio fichero).
- Workflow en `.github/workflows/`. Copiar del enunciado nombres de jobs, actions, versiones y comandos.
- Subir workflow **y** los archivos que necesita. Revisar la pestaña Actions hasta que esté en verde.